MOB CONTROL

Gameplay deconstruction + recommended first web version | Design and engineering reference | November 2026

Executive recommendation: Prototype a one-screen, 45-75 second battle: drag a horizontally moving cannon, shoot blue mobs through two multiplier gates, hold back continuously spawning red mobs, and destroy one enemy base. Keep one-touch controls, instant retry, and a tiny permanent upgrade between battles. Do not build the full metagame first.

1. The game in one minute

Mob Control is a hybrid-casual crowd-combat game. The player feeds an automatic stream of blue units through multiplication gates. Red units move down the battlefield from enemy defences. Crowds collide, and surviving blue mobs damage enemy structures. The battle ends when the required enemy base is defeated or the enemy reaches the player-side cannon.

Moment-to-moment: aim cannon -> multiply units -> contest enemy waves -> overwhelm defences -> destroy base -> collect rewards -> upgrade -> next battle.

2. Battlefield and level structure

The classic battlefield is a narrow, vertically oriented arena with the cannon at the near end, gates and obstacles in the middle, and enemy spawners or bases toward the far end. The visual perspective is elevated, near top-down. Some versions or maps have several sequential stages: after clearing a defence, the cannon/camera advances to another section.

Element	Player-facing function
Cannon	Slides left/right; fires a stream of allied mobs toward the enemy.
Multiplier gate	A mob entering an xN gate becomes N mobs total, creating exponential-looking growth.
Moving gate / obstacle	Adds timing and aiming choices; walls constrain lanes and reduce efficient paths.
Enemy spawner / tower	Creates red mobs that advance toward the player; structures can be attacked and destroyed.
Final enemy base	Main victory target and source of end-of-battle payoff.
Champion / ability	Optional later-layer power spike; not required to validate the core loop.

Level variety: gate placement, motion, barrier configuration, number and placement of spawners, enemy pressure, multi-stage layout, and special enemies. These are knobs, not necessarily handcrafted numbered levels.

3. Inputs, spawning and combat

• Input: horizontal dragging aims the cannon. Units move and attack automatically; touch-to-fire versus autofire differs between versions.

• Enemy flow: active enemy sources repeatedly create red mobs. The opposing crowd advances toward the player end, exerting continuous pressure.

• Mob combat: units collide and fight at close range; basic mobs appear expendable while upgraded units and Champions can use different stats.

• Gate effect: crossing a multiplier trigger spawns additional friendly units, with positions spread to avoid a single stacked point.

• Structure damage: surviving blue mobs that reach a hostile base or tower deplete its health/counter until it is destroyed.

The proprietary engine and exact physics/AI rules are not public. The behaviour above describes observable gameplay; the engineering approach later in this document is a recommended approximation, not a source-code claim.

4. Win, lose and what happens next

Situation	Outcome
Win	Destroy the required enemy final base or finish the final battle stage.
Lose	A red enemy breaks through to the player cannon/end zone in the classic mode.
After win	Show loot such as coins and bricks, with optional bonuses depending on mode/version.
Between battles	Spend coins to improve combat cards; use bricks to progress base construction; continue to another battle.
After loss	Show a clear failure state and offer retry; permanent progression should not be erased.

5. Why it retains players

• Every few seconds: watch a small stream turn into a large crowd after a multiplier gate.

• Every minute: finish a battle with an obvious win/lose resolution and reward burst.

• Across sessions: improve cannon/mobs, accumulate construction progress, collect new cards or compete in events.

• Decision density: one simple drag does two jobs at once: attack enemy defences and defend against red breakthroughs.

6. Recommended browser MVP - strict scope

Design goal: prove the central skill/reward loop works in a mobile browser and on desktop mouse before investing in base construction, card inventories, ad economy, elaborate animations or content-heavy level creation.

Ship these features

System	MVP decision
Camera / arena	Fixed portrait-friendly battlefield; one screen, no scrolling.
Player	One cannon at bottom; drag horizontally within rails; automatic shots while playing.
Mobs	One blue type and one red type; blue/red differ only by colour and direction.
Gates	Two static gates x2 and x3, optionally one gate moving side to side in later test.
Enemies	One spawner produces reds in repeating waves until destroyed.
Objective	One enemy base with visible health; win at zero HP.
Lose condition	Any red mob reaches the bottom danger line/cannon area.
Post-battle	Victory burst, coin reward, replay/next button.
Permanent upgrade	Choose one: cannon fire rate level 1-5; increase with earned coins.
Content	Three parameter presets reusing the same arena.

Do not build in version one

No Champions, no card packs, no base-building islands, no league system, no procedural maps, no complex enemy AI, no multiple ammunition types, no online leaderboard, no ad monetisation dependency, and no physics engine per soldier.

7. Proposed 60-second battle (tunable)

Moment	What player sees	Why it matters
0-5s	Cannon already shooting; x3 gate visibly better than x2.	Instantly understandable action.
5-15s	First blue wave multiplies; red mobs emerge.	First payoff and first threat.
15-35s	Front line fluctuates; aim adjustments change advantage.	A repeatable skill decision.
35-55s	Blue mobs begin damaging the base.	Clear endgame progress.
55-75s	Base falls or enemy breaches; reward/retry.	Strong, quick resolution.

These times are a design target, not Mob Control measurements. Tune with actual playtests. If most first-time players lose before understanding gates, reduce early red pressure; if players can ignore aiming, increase the gate-selection advantage.

8. Concrete starter values (prototype assumptions)

Parameter	Starting value
Arena width	10 world units (2D X/Z simulation).
Player firing	5 blue mobs/sec; continuous.
Blue speed	4.2 units/sec forward.
Red movement	3.2 units/sec toward cannon.
Enemy spawn	2 red mobs/sec initially; increase gradually to 3/sec.
Gate values	x2 at left lane; x3 at right lane.
Cannon X range	-4.2 to +4.2 world units.
Basic combat	On contact, remove one blue and one red mob.
Base HP	75 one-hit mob impacts, then tune.
Max active units	First target 300-500 visible mobs; optimize later.
Reward / upgrade	100 coins win, 50 coins first fire-rate upgrade.

Balancing warning: these numbers are illustrative placeholders. A working crowd battle depends on unit travel times, gate geometry and enemy throughput together, so tune as a system rather than treating these as canonical values.

9. Physics and rendering architecture for Pixi + Three.js

Use a 2D simulation on the X/Z plane even when rendering the characters in Three.js. Avoid rigid bodies for each mob. Represent mobs in typed arrays or pooled structures; draw crowds with InstancedMesh. Use simple deterministic steering and approximate proximity combat.

Part	Recommended approach
Simulation	Fixed-step 30 or 60 Hz update, separate from rendering delta.
Forward movement	Blue mobs seek top direction; red mobs seek bottom direction.
Avoidance	Cheap local separation using a uniform spatial hash grid.
Collisions	Spatial grid query, small-radius overlap, resolve deaths and base hits.
Gates	Plane/rectangle entry trigger with per-unit gate IDs to avoid repeated multiplication while overlapping.
Rendering	Instanced meshes (or Pixi sprites for flatter art), with interpolation.
Pooling	Reuse mobs and instances to prevent GC spikes on mobile.
Scaling	Cap unit population and bound processing; tune visual density separately.

Suggested data model

BattleConfig: arena bounds, cannon firing rate, gate positions/values, spawner rate, base HP, mob speeds, win/lose triggers, and reward. **BattleRuntime**: active mobs, tower HP, elapsed time, stage state, and selected upgrade levels.

Common implementation pitfalls

• Gate overlap accidentally multiplies a mob on every frame; use entry/crossing detection.

• Thousands of object allocations cause frame stutter; pool units and keep update loops lean.

• Crowd clustering hides the player decision; ensure aim through x3 visibly outperforms x2.

• Enemy waves that arrive too quickly create surprise losses; signal the threat near the bottom.

• Lack of reward feedback makes rounds feel interchangeable; show a strong base-break and upgrade effect.

10. Development order and success criteria

Pass	Build	Acceptance check
A. Feel	Cannon, one gate, blue mobs, base damage.	Within 5 seconds, a new player understands what to do.
B. Challenge	Red spawner, combat, defeat, restart.	Aiming changes whether the player wins.
C. Satisfaction	x2/x3 choices, hit feedback, end-of-round celebration.	Crowd multiplication and base destruction feel rewarding.
D. Retention	3 presets, coin reward, 1 upgrade.	Users voluntarily begin another round.
E. Optimise	Pooling, instancing, mobile input and memory tuning.	Stable frame times on target phones.

Playtest events to instrument

Track battle_start, first_gate_crossed, first_enemy_contact, first_base_damage, battle_win, battle_lose (with reason), battle_duration, restart, next_battle, upgrade_purchase, active_unit_peak and frame_time_percentiles. Segment first-session players separately from repeats.

Go / no-go decision: If players understand the gate choice, feel a clear improvement after upgrading, and start a second battle without prompting, add more level layouts. If not, adjust combat visibility and feedback before expanding features.

11. What to build next - only after the MVP works

Iteration 2: moving gates, 1-2 obstacle types, a second enemy tower, another cannon upgrade. Iteration 3: short two-stage levels with a camera advance, a special enemy or mini-boss, and a simple unlock map. Later: Champion abilities and a meta construction layer if analytics justify the extra complexity.

Version caveat: This document combines widely observable classic Mob Control mechanics with a deliberately simplified proposed implementation. Exact spawning rates, internal physics, map selection, economy and newly introduced game modes may differ by release.